SCCAD 3nm PDK

User Manual

PDK Development and Modification

Version	v1.0
Release Date	TBD
Organization	SCCAD Lab, University of Southern California
Contact	sw.jung@gatech.edu (Sungwoo Jung)

This manual describes the full **SCCAD 3nm PDK** development flow and provides step-by-step guidance for modifying key PDK components, including SPICE model cards and interconnect models (ITF/ICT). It covers the tools and scripts used in each stage of the development pipeline, serving as a reference for users who wish to customize or extend the PDK.

1 Overall Flow of PDK Development

The SCCAD-3nm PDK was developed through a structured multi-stage flow integrating TCAD device simulation, physical cell design, parasitic extraction, and library characterization. The full development pipeline is illustrated in **Figure 1**. The yellow-highlighted boxes in the figure represent the final PDK deliverables consumed by the PnR flow.

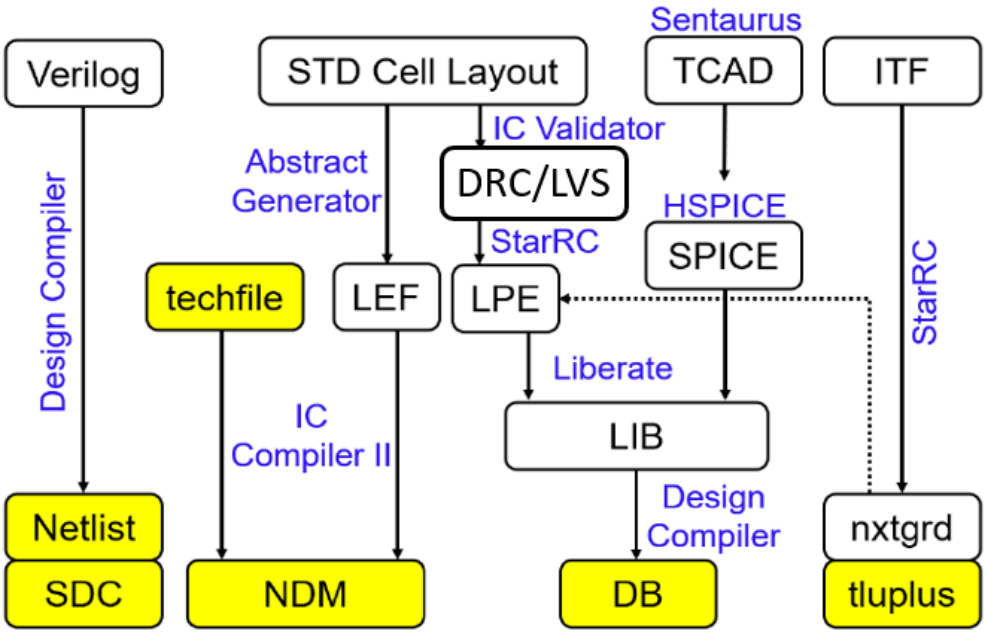

Figure 1 Overall Flow of PDK Development

1.1 EDA Tools Used

The SCCAD-3nm PDK development flow relies on both Synopsys and Cadence EDA tools. The table below summarizes all tools used, organized by vendor and stage.

Vendor	Steps	Tool	Description
Synopsys	TCAD modeling	Synopsys-Sentaurus	3nm Si GAAFET Model
	Compact modeling	Synopsys-HSPICE	Generate BSIM-CMG model
	STD Cell Layout	Synopsys-Custom Compiler	Generate Cell layouts
	LVS/DRC	Synopsys-ICV	Conduct LVS and DRC
	LPE	Synopsys StarRC	Extract RC information of STD cells
	nxtgrd/tluplus gen.	Synopsys-StarRC	Convert ITF into TLUPLUS and NXTGRD
	NDM generation	Synopsys-ICC2	Generate .ndm
	DB Generation	Synopsys-Design Compiler	Convert .lib into .db
	Logic Synthesis	Synopsys-Design Compiler	Generate .netlist and .sdc
Cadence	LEF generation	Cadence-Abstract Generator	Generate Abstract and LEF
	LIB generation	Cadence-Liberate	Generate LIB

1.2 Stage-by-Stage Description

Stage 1 — Device Modeling: TCAD to SPICE

PDK development begins with **TCAD simulation** using Synopsys Sentaurus to characterize the 3nm GAAFET device structure. The simulated device characteristics (Id-Vgs, Id-Vds curves) are fitted to a BSIM-CMG compact model, producing the **SPICE model card** that accurately captures transistor behavior across process, voltage, and temperature (PVT) corners. The SPICE models are subsequently used in two downstream paths: cell-level LPE simulation and Liberty characterization.

Stage 2 — Physical Cell Design: Layout to LEF/NDM

Standard cell layouts are drawn in compliance with the 3nm design rules and verified through **DRC/LVS** using Synopsys IC Validator. Once sign-off clean, Cadence **Abstract Generator** extracts the cell abstract view — pin locations, obstruction layers, and cell dimensions — to produce the **LEF** file. In parallel, the physical layout combined with the technology file (techfile) is compiled into the ICC2-native **NDM** reference library using IC Compiler II, which packages all physical views required for PnR into a single database.

Stage 3 — Parasitic Extraction: LPE and RC Models

Layout Parasitic Extraction (**LPE**) is performed on each standard cell using **StarRC**, driven by the ITF interconnect technology file. The extracted parasitic netlist captures the RC contributions of local interconnect within each cell. In addition, StarRC processes the ITF to generate the **nxtgrd** and **TLUPlus** RC tables, which are used during full-chip PnR for fast, field-solver-calibrated wire parasitic estimation.

Stage 4 — Library Characterization: LPE to LIB/DB

Cell timing and power characterization is performed by Cadence **Liberate** using HSPICE simulation. Each cell is simulated with the extracted parasitic netlist (LPE) and the corresponding SPICE model card, sweeping input slew and output load conditions across all PVT corners. The resulting Non-Linear Delay Model (**NLDM**) data is written out as a **Liberty (.lib)** file. The Liberty file is then compiled into binary **.db** format using Synopsys Design Compiler for use in synthesis and sign-off.

Stage 5 — PDK Deliverables

The outputs of the development flow constitute the complete set of PDK files required to run a full RTL-to-GDS flow:

- **Netlist/SDC** — Gate-level netlist and timing constraints from synthesis
- **NDM** — ICC2-native reference library for physical implementation
- **LEF** — Cell abstract views for Cadence Innovus
- **Liberty (.lib / .db)** — Timing and power data for synthesis and sign-off
- **TLUPlus / nxtgrd** — RC tables for Synopsys StarRC parasitic extraction
- **QRC tech file (.tch)** — RC model for Cadence Quantus parasitic extraction

Any modification to an upstream component (e.g., SPICE model or ITF) requires re-running all downstream steps to ensure consistency across the PDK. The following chapter describes how to modify the two most commonly updated components: the SPICE model and the interconnect model.

2 PDK Component Modification

2.1 SPICE Model

SPICE model cards define the electrical behavior of transistors used in circuit simulation and cell characterization. The SCCAD-3nm PDK provides HSPICE-format model cards for both NMOS and PMOS devices across five PVT corners. Modifying the SPICE model is necessary when the target process parameters change — for example, when exploring a different threshold voltage flavor, adjusting the device geometry, or fitting the model to a new set of TCAD simulation data.

File location:

```
$PDK_ROOT/model/hspice/
├── sccad3nm_tt.l   # Typical-Typical (TT) corner
├── sccad3nm_ss.l   # Slow-Slow (SS) corner
├── sccad3nm_ff.l   # Fast-Fast (FF) corner
├── sccad3nm_fs.l   # Fast-Slow (FS) corner
└── sccad3nm_sf.l   # Slow-Fast (SF) corner
```

Challenge: Manual Parameter Tuning

A BSIM-CMG model card for a 3nm GAAFET device contains tens of interdependent parameters — including threshold voltage (**phig**), carrier mobility (**u0, ua, etamob**), saturation velocity (**vsat**), and short-channel effect coefficients (**eta0, pdibl2, dvtp0**), among others. Manually tuning each parameter by trial and error to match a target Id-Vgs / Id-Vds characteristic is extremely time-consuming and prone to inconsistency, particularly when fitting across multiple bias conditions simultaneously.

Solution: Automated Curve Fitting Script

To address this, the SCCAD-3nm PDK includes an automated curve fitting script (**auto_curve_fitting.sp**) that leverages HSPICE's built-in optimization engine (**opt2**) to extract all model parameters simultaneously by minimizing the error between simulated and reference Id-V curves. The reference data — obtained from TCAD simulation or measurement — is supplied as a tabulated data file (**3nm_Ref_iv.dat**). The optimizer iterates up to 5000 times with tight convergence tolerances, typically converging within minutes.

Script structure: auto_curve_fitting.sp

The script defines the device under test with the target geometry:

```
* Device under test — NMOS LVT, W=20nm, L=12nm, nfin=1
m1 drain gate 0 bulk nmos_lvt w=2e-8 l=1.2e-8 nfin=1
```

Each parameter is declared with an initial value and a search range using the **opt2(initial, min, max)** syntax:

```
.param
+ phig    = opt2(4.5,        4.4,     4.8  )  $ Gate work function (eV)
+ u0      = opt2(0.025145,   0.005,   0.03 )  $ Low-field mobility
+ etamob  = opt2(3.0535,     2.7,     3.3  )  $ Mobility degradation exponent
+ vsat    = opt2(1.1e+5,     1.0e+5,  1.3e+5) $ Saturation velocity (cm/s)
+ eta0    = opt2(1.3571,     1.0,     3.0  )  $ DIBL coefficient
+ pdibl2  = opt2(6.4836e-3,  2e-3,    1e-2 )  $ DIBL effect parameter
+ dvtp0   = opt2(-1.2334e-2,-0.013,  -0.01)   $ Vth shift parameter
+ ...                                          $ (16 parameters total)
```

The optimization minimizes the RMS error between simulated current and the reference data:

```
* Load reference IV data (TCAD or measurement)
```

```
.data iv merge
+ file=3nm_Ref_iv.dat  vds=1  vgs=2  ids=3
.enddata

* Run optimization — up to 5000 iterations, tight convergence
.dc data=iv optimize=opt2 results=compl2 model=optmod2
.model optmod2 opt itropt=5000 relout=1e-6 relin=1e-8 close=10

* Error metric: RMS error between reference and simulated Id
.measure dc compl2 err2 par(ids) i(m1) minval=1e-11 ignore=1e-12
```

How to use the script:

1. Prepare **3nm_Ref_iv.dat** with columns [Vds, Vgs, Ids] from TCAD simulation or target specification.
2. Adjust the initial values and search ranges in the **.param** block if needed, based on prior knowledge of the target process.
3. Run the script with HSPICE: **hspice auto_curve_fitting.sp > fitting.log**
4. Copy the optimized parameter values from the log file into the target corner model card (e.g., **sccad3nm_tt.l**).
5. Verify the fitted model by re-running simulation without optimization and comparing Id-Vgs/Id-Vds curves against the reference data.
6. If the change affects cell timing or power, re-run Liberty characterization with Liberate and recompile the **.db** file before the next PnR run.

2.2 Interconnect Model (ITF — Synopsys Flow)

The Interconnect Technology File (**ITF**) describes the physical properties of the metal stack — layer thicknesses, sheet resistances, via resistances, and dielectric constants — used by StarRC to compute parasitic resistance and capacitance. Accurate RC models are critical for timing convergence at 3 nm. ITF modification is necessary when exploring alternative metal stack configurations, modeling process variation, or calibrating the PDK to a new foundry target.

File location:

```
$PDK_ROOT/ITF/
├── sccad3nm.itf        # Main Interconnect Technology File
├── sccad3nm.tluplus    # TLUPlus RC table (used by ICC2)
└── sccad3nm.nxtgrd     # nxtgrd RC table (used by StarRC)
```

ITF key parameters:

```
ILAYER  M1
  RTHICK    = [value]    ; Metal thickness (nm)
  RHO       = [value]    ; Sheet resistance (Ohm/sq)
  WMIN      = [value]    ; Minimum wire width (nm)
  ...

ILAYER  VIA1
  RPV       = [value]    ; Via resistance (Ohm)
  ...

IDIELECTRIC
  ER        = [value]    ; Relative permittivity (k)
  ...
```

Challenge: Consistent Resistance Scaling

A full 3nm BEOL stack contains metal layers (M0–M9, MBPR, MB1–MB2) and via layers (V0–V8, VBPR, VB1, μ-TSV, BSC), each with its own resistance entry in the ITF. When a process change requires scaling resistance values — for example, modeling a ±10% resistance variation — manually editing every **RHO**, **RPV**, and **VALUES** entry across the file is tedious and error-prone.

Solution: Automated ITF Resistance Scaling Script

The PDK provides a Python utility script (**auto_itf_scaling.py**) that automates resistance scaling across the entire ITF in a single command using regex-based replacements. Layer thickness and dielectric constants are left unchanged.

Script usage:

```
python auto_itf_scaling.py \
  -i sccad3nm.itf          \  # Input ITF file
  -o sccad3nm_scaled.itf   \  # Output ITF file
  -s 1.5                   \  # Scale factor (e.g., 1.5x)
  -t all                      # Target: via | metal | table | all
```

The script applies three independent regex-based replacements:

```
# Scale via resistance: RPV=value -> RPV=value*scale
if ('via' in targets or scale_all) and 'RPV=' in line:
    line = re.sub(r'RPV=([0-9\.eE+-]+)',
                  lambda m: f"RPV={float(m.group(1))*scale_factor:.6g}",
                  line)

# Scale sheet resistance: RHO=value -> RHO=value*scale
```

```
if ('metal' in targets or scale_all) and 'RHO=' in line:
    line = re.sub(r'RHO=([0-9\.eE+-]+)',
                  lambda m: f"RHO={float(m.group(1))*scale_factor:.6g}",
                  line)

# Scale resistance table: VALUES { v1 v2 ... } -> VALUES { v1*s v2*s ... }
if ('table' in targets or scale_all) and 'VALUES' in line and '{' in line:
    line = re.sub(r'VALUES\s*\{\s*(.*?)\s*\}',
                  lambda m: "VALUES { " +
                  "\t".join(f"{float(v)*scale_factor:.6g}"
                            for v in m.group(1).split()) + " }",
                  line)
```

How to use the script (Synopsys flow):

7. Select the target with **-t via**, **-t metal**, **-t table**, or **-t all**.
8. Run the script, e.g.: **python auto_itf_scaling.py -i sccad3nm.itf -o sccad3nm_110pct.itf -s 1.1 -t metal**
9. Verify the output file — RHO/RPV values updated, thickness and dielectric constants unchanged.
10. Re-generate the TLUPlus and nxtgrd RC tables using StarRC: **StarXtract -cmd scripts/starrc/gen_tluplus.cmd**
11. Re-run the ICC2 PnR flow with the new RC tables and compare PPA results against the baseline.

2.3 Interconnect Model (ICT — Cadence Flow)

In the Cadence Quantus parasitic extraction flow, the interconnect technology is described using the **ICT (Interconnect Characterization Technology)** file format — the Cadence-native equivalent of the Synopsys ITF. The ICT file encodes the physical and electrical properties of every BEOL layer. Quantus consumes the ICT file to generate the **QRC technology file (.tch)**, which is used for full-chip RC extraction during the Innovus PnR flow.

File location:

```
$PDK_ROOT/ICT/
├── sccad3nm.ict       # Main ICT file for Quantus
└── sccad3nm.tch       # Generated QRC technology file
```

ICT file structure:

```
Layer M1 {
  Conductor {
    Resistivity  = [value]   # Sheet resistance (Ohm/sq)
    Thickness    = [value]   # Metal thickness (nm)
    Width        = [value]   # Minimum width (nm)
  }
  Dielectric {
    Permittivity = [value]   # Relative permittivity (k)
    Thickness    = [value]   # ILD thickness (nm)
  }
}

Via VIA1 {
  Resistance   = [value]     # Via resistance (Ohm)
  Width        = [value]     # Via width (nm)
  Height       = [value]     # Via height (nm)
}
```

Challenge: Consistent Resistance Scaling

The ICT file contains **Resistivity** (metal sheet resistance) and **Resistance** (via resistance) entries for every layer in the 3nm BEOL stack. When a process change requires globally scaling these values, manually editing each entry is tedious and error-prone — particularly given the structural differences between the ICT block-based format and the ITF key=value format.

Suggested Solution: Automated ICT Scaling Script

The SCCAD-3nm PDK does not currently include a dedicated ICT scaling script, but one can be developed following the same design principles as **auto_itf_scaling.py**. The core approach — using Python regular expressions to locate and scale specific numeric fields — translates directly to the ICT format with minor syntax adjustments. The two target fields are:

- **Resistivity** — Metal sheet resistance within each Conductor { } block. Analogous to RHO= in the ITF.
- **Resistance** — Via resistance within each Via { } block. Analogous to RPV= in the ITF.

Suggested script structure (auto_ict_scaling.py):

```
import re, argparse

def scale_ict_resistance(input_file, output_file, scale_factor, targets):
    with open(input_file, 'r') as f:
        lines = f.readlines()

    updated = []
    for line in lines:
        # Scale metal sheet resistance (Resistivity field)
        if ('metal' in targets or 'all' in targets):
            line = re.sub(
                r'(Resistivity\s*=\s*)([0-9\.eE+-]+)',
                lambda m: m.group(1) + f"{float(m.group(2))*scale_factor:.6g}",
                line)
        # Scale via resistance (Resistance field)
        if ('via' in targets or 'all' in targets):
            line = re.sub(
                r'(Resistance\s*=\s*)([0-9\.eE+-]+)',
                lambda m: m.group(1) + f"{float(m.group(2))*scale_factor:.6g}",
                line)
        updated.append(line)

    with open(output_file, 'w') as f:
        f.writelines(updated)
```

Suggested workflow after ICT modification:

12. Develop **auto_ict_scaling.py** using the regex pattern from auto_itf_scaling.py as a reference, adapting field names to the ICT format (Resistivity, Resistance).
13. Run the script: **python auto_ict_scaling.py -i sccad3nm.ict -o sccad3nm_scaled.ict -s 1.1 -t all**
14. Verify the output — confirm Resistivity and Resistance values are scaled while Thickness and Permittivity remain unchanged.
15. Re-generate the QRC technology file from the modified ICT using Quantus: **quantus -cmd scripts/quantus/gen_qrc.cmd**
16. Re-run the Innovus PnR flow with the new QRC tech file and compare PPA results against the baseline.